

BVMT AI Platform — Sprint 2

Days 11 · 12 · 13 — Feature Engineering + TFT Training
Esprit School of Business — Tunisia | March 2026

Where You Are Before Starting

Sprint 1 is complete. The financial extraction batch is still running in the background — that is fine, it does not block Sprint 2. Here is your exact status:

Component	Status	Sprint 2 Impact
daily_prices	~120,000 rows, 68 stocks ✓	Main TFT input
computed_indicators	MA5/20/50, RSI14, volatility ✓	TFT features
company_metadata	68 stocks, types/sectors ✓	Static features
financial_statements	~350+ records, batch running	Not needed yet
TFT model	Not started — Days 11-13 task	Build now

Why we can start Sprint 2 now even though the financial batch is not done

The TFT model only uses price data + technical indicators (daily_prices + computed_indicators).
Financial ratios are used by FundamentalAgent in Sprint 3 — not by TFT.
You have all the data TFT needs. The batch running in the background does not block you.

DAY 11 — Feature Engineering — Build TFT Input Dataset (3-4 hours)

Day 11 is entirely local — no GPU, no Lightning AI, no API calls. You query your PostgreSQL database, combine three tables into one flat dataset, compute the prediction target, and export a CSV. This CSV is what goes into the TFT model on Day 13.

What is Feature Engineering?

The TFT model cannot read your database directly. It needs a single flat table where each row is one stock on one day, with all the features (MA, RSI, etc.) and the target variable (did the price go up in 7 days?) already computed. Feature engineering is the process of building that table.

Think of it like this: your database has the raw ingredients. Feature engineering is the cooking — combining and transforming those ingredients into the exact format the model needs.

Step 1 — Create the notebook

In VS Code, create a new Jupyter notebook at:

```
notebooks/feature_engineering.ipynb
```

This notebook has 8 cells. Each cell does one specific thing. Run them in order.

Step 2 — Cell by cell instructions

Cell 1 — Imports and database connection

This cell loads all the Python libraries you need and connects to your PostgreSQL database.

```
import pandas as pd
import numpy as np
import psycopg2
import os
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

def get_conn():
    return psycopg2.connect(
        host=os.getenv('DB_HOST'),
        port=int(os.getenv('DB_PORT', 5432)),
        dbname=os.getenv('DB_NAME'),
        user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD')
    )

print('Connection ready')
```

Cell 2 — Load daily_prices from PostgreSQL

This loads ALL price data for all 68 stocks. The WHERE clause filters to Groupe 11 only (main market stocks). This query takes about 10-20 seconds.

```
conn = get_conn()

df_prices = pd.read_sql('''
    SELECT
        dp.seance,
        dp.isin_code,
        dp.ticker,
        dp.ouverture      AS open_price,
        dp.cloture        AS close_price,
        dp.plus_bas        AS low_price,
        dp.plus_haut       AS high_price,
        dp.quantite_negocie AS volume,
        dp.nb_transaction AS num_trades
    FROM daily_prices dp
    ORDER BY dp.ticker, dp.seance
''', conn)

conn.close()

# Convert seance to datetime
df_prices['seance'] = pd.to_datetime(df_prices['seance'])

print(f'Loaded: {len(df_prices):,} rows, {df_prices["ticker"].nunique()} stocks')
print(f'Date range: {df_prices["seance"].min()} to {df_prices["seance"].max()}')
df_prices.head()
```

Expected output: ~120,000 rows, 68 stocks, 2016-01-01 to 2025-12-31

Cell 3 — Load computed_indicators and join

This loads the technical indicators (MA, RSI, volatility) and joins them to the price data. The join key is (seance, isin_code) — same date, same stock.

```
conn = get_conn()

df_ind = pd.read_sql('''
    SELECT
        seance,
        isin_code,
        ma_5,
        ma_20,
        ma_50,
        rsi_14,
        daily_return,
        volatility_20,
        volume_ma_20
    FROM computed_indicators
    ORDER BY isin_code, seance
''', conn)
```

```

conn.close()

df_ind['seance'] = pd.to_datetime(df_ind['seance'])

# Join prices with indicators on seance + isin_code
df = df_prices.merge(df_ind, on=['seance', 'isin_code'], how='inner')

print(f'After join: {len(df):,} rows')
print(f'Columns: {list(df.columns)}')

```

The inner join means we only keep rows that exist in BOTH tables. Some early dates may be dropped because indicators need 50 days of history to compute MA50.

Cell 4 — Load company_metadata and add static features

The TFT model needs static features — things that do not change over time like company type and sector. We get these from company_metadata.

```

conn = get_conn()

df_meta = pd.read_sql('''
    SELECT
        isin_code,
        ticker,
        groupe
    FROM company_metadata
''', conn)

conn.close()

# Add company_type based on ticker name patterns
# This is a simple rule — you can improve it later
BANK_TICKERS = {
    'AMEN BANK', 'BIAT', 'BNA', 'STB', 'BH', 'ATTIJARI BANK',
    'ATB', 'UBCI', 'UIB', 'BT', 'WIFACK INT BANK', 'BNA'
}

INSURANCE_TICKERS = {
    'STAR', 'ASTREE', 'BH ASSURANCE', 'ASSUR MAGHREBIA',
    'ASSU MAGHREBIA VIE', 'TUNIS RE', 'BNA ASSURANCES'
}

def get_type(ticker):
    if ticker in BANK_TICKERS: return 'bank'
    if ticker in INSURANCE_TICKERS: return 'insurance'
    return 'non_bank'

df_meta['company_type'] = df_meta['ticker'].apply(get_type)

# Join metadata to main dataframe
df = df.merge(df_meta[['isin_code', 'company_type']], on='isin_code', how='left')

print(f'Company type distribution:')
print(df.groupby('company_type')['ticker'].nunique())

```

Cell 5 — Compute the TARGET variable (most important step)

The target variable is what the TFT model learns to predict. It answers: will this stock's closing price be higher in 7 trading days compared to today?

This is a binary classification: 1 = price goes UP, 0 = price goes DOWN or stays flat.

```
# Sort by ticker and date — CRITICAL, must be sorted for shift() to work correctly
df = df.sort_values(['ticker', 'seance']).reset_index(drop=True)

# For each stock, look 7 rows ahead to get the future closing price
# shift(-7) means: take the value from 7 rows later IN THE SAME GROUP
df['future_price'] = df.groupby('ticker')['close_price'].shift(-7)

# Target: 1 if future price is higher than today's price, 0 otherwise
df['direction_7d'] = (df['future_price'] > df['close_price']).astype(int)

# Remove the last 7 rows per stock — they have no future price to predict
df = df.dropna(subset=['direction_7d', 'future_price'])

# Check class balance — important for model training
balance = df['direction_7d'].value_counts(normalize=True)
print(f'Class balance:')
print(f'  UP    (1): {balance.get(1,0)*100:.1f}%')
print(f'  DOWN  (0): {balance.get(0,0)*100:.1f}%')

if balance.get(1, 0) > 0.60:
    print('WARNING: Class imbalance > 60/40 — use class_weight in training')
else:
    print('Class balance is acceptable')
```

Why shift(-7) and not shift(7)?

shift(-7) moves backwards: it gives you the value 7 rows AHEAD in the future.

shift(7) moves forwards: it gives you the value 7 rows BEHIND in the past.

We want to predict the FUTURE, so we use shift(-7).

After shift(-7), the last 7 rows per stock will have NaN — we drop those with dropna().

Cell 6 — Encode tickers as integers + create time index

The TFT model requires numeric IDs for groups (stocks) and a numeric time index. It cannot work with string values like 'AMEN BANK' or dates like '2016-01-04' directly.

```
# Encode ticker as integer ID (0, 1, 2, ... 67)
# factorize() assigns a unique integer to each unique ticker string
df['ticker_id'], ticker_mapping = pd.factorize(df['ticker'])

# Save the mapping so you can decode predictions later
# ticker_mapping[0] = 'AMEN BANK', ticker_mapping[1] = 'ARTES', etc.
ticker_map_df = pd.DataFrame({
    'ticker_id': range(len(ticker_mapping)),
    'ticker': ticker_mapping
})
```

```

# Encode company_type as integer
type_map = {'bank': 0, 'insurance': 1, 'non_bank': 2}
df['company_type_id'] = df['company_type'].map(type_map)

# Create time_idx: integer days since 2016-01-01
# TFT needs a continuously increasing integer as time index
origin = pd.Timestamp('2016-01-01')
df['time_idx'] = (df['seance'] - origin).dt.days

print(f'Ticker IDs: {df["ticker_id"].nunique()} unique')
print(f'Time index range: {df["time_idx"].min()} to {df["time_idx"].max()}')
print()
print('Ticker ID mapping (first 10):')
print(ticker_map_df.head(10))

```

Cell 7 — Filter, clean, and export

Remove stocks with insufficient history, drop rows with NULL features, and export to CSV. This CSV is what you upload to Lightning AI.

```

# Keep only stocks with at least 500 trading days of history
# (~2 years minimum - stocks with less data are too sparse for TFT)
stock_counts = df.groupby('ticker')['seance'].count()
valid_tickers = stock_counts[stock_counts >= 500].index
df = df[df['ticker'].isin(valid_tickers)]
print(f'Stocks with 500+ days: {len(valid_tickers)}')

# Select final columns for the model
FEATURE_COLS = [
    'time_idx',          # time index (required by TFT)
    'ticker_id',         # group ID (required by TFT)
    'seance',            # actual date (for reference)
    'ticker',            # ticker string (for reference)
    'isin_code',         # ISIN (for reference)
    'company_type_id',   # static feature
    'close_price',       # main price feature
    'volume',            # trading volume
    'daily_return',      # % change from yesterday
    'ma_5',              # 5-day moving average
    'ma_20',             # 20-day moving average
    'ma_50',            # 50-day moving average
    'rsi_14',            # RSI momentum indicator
    'volatility_20',     # 20-day rolling volatility
    'volume_ma_20',     # 20-day average volume
    'direction_7d',     # TARGET: 1=up, 0=down
]

df_final = df[FEATURE_COLS].dropna()

# Create output folder
output_dir = Path(r'C:\Users\Negza\Desktop\projects\pfe\bvmt_project\data\features')
output_dir.mkdir(parents=True, exist_ok=True)

# Export main dataset
csv_path = output_dir / 'tft_features.csv'
df_final.to_csv(csv_path, index=False)

```

```
# Export ticker mapping separately
ticker_map_df.to_csv(output_dir / 'ticker_mapping.csv', index=False)

print(f'Exported: {len(df_final):,} rows x {len(FEATURE_COLS)} columns')
print(f'File size: {csv_path.stat().st_size / 1024 / 1024:.1f} MB')
print(f'Saved to: {csv_path}')
```

Cell 8 — Verification and class balance report

Always verify your dataset before training. These checks catch problems before you waste GPU time.

```
print('='*50)
print('DATASET VERIFICATION REPORT')
print('='*50)

print(f'Total rows      : {len(df_final):,}')
print(f'Unique stocks     : {df_final["ticker_id"].nunique()}'')
print(f'Date range       : {df_final["seance"].min()} to {df_final["seance"].max()}')
print(f'NULL values       : {df_final.isnull().sum().sum()}')

print()
print('Rows per stock (sample):')
print(df_final.groupby('ticker')['time_idx'].count().describe())

print()
print('Class balance (direction_7d):')
bal = df_final['direction_7d'].value_counts(normalize=True)
print(f'  UP  (1)  : {bal.get(1,0)*100:.1f}%')
print(f'  DOWN (0) : {bal.get(0,0)*100:.1f}%')

print()
print('Feature statistics:')
print(df_final[['close_price', 'rsi_14', 'ma_20', 'volatility_20']].describe().round(2))
```

What to do with the class balance result

If UP=55%, DOWN=45% → perfectly balanced, no action needed.
 If UP=62%, DOWN=38% → imbalanced. In train_tft.py set class_weight='balanced'.
 If UP=70%, DOWN=30% → heavily imbalanced. Consider using only 2020-2025 data
 (post-COVID recovery creates artificial upward bias in older data).

DAY 12 — Prepare Training Script + Upload to Lightning AI

(2-3 hours)

Day 12 is preparation day. You write the TFT training script, set up your Lightning AI studio, and upload everything. No training happens yet — training runs on Day 13. Think of today as packing your bags before a flight.

Part A — Write training/train_tft.py (1.5 hours)

This file already exists as a skeleton in your project. You are now filling it in with real code. Open training/train_tft.py in VS Code and replace its contents with the following:

The complete train_tft.py file

```
# training/train_tft.py
#
# TFT (Temporal Fusion Transformer) training script
# Runs on Lightning AI free T4 GPU
# Predicts 7-day stock direction for BVMT stocks

import pandas as pd
import numpy as np
import torch
from pathlib import Path
from pytorch_forecasting import TemporalFusionTransformer, TimeSeriesDataSet
from pytorch_forecasting.metrics import CrossEntropy
from pytorch_lightning import Trainer
from pytorch_lightning.callbacks import EarlyStopping, ModelCheckpoint
import warnings
warnings.filterwarnings('ignore')

# — Configuration —————
DATA_PATH = Path('data/features/tft_features.csv')
MODEL_DIR = Path('models')
MODEL_DIR.mkdir(exist_ok=True)

MAX_ENCODER_LENGTH = 60 # how many past days TFT sees (60 trading days = 3 months)
MAX_PREDICTION_LENGTH = 7 # how many days ahead we predict (1 week)
BATCH_SIZE = 128 # samples per training batch
MAX_EPOCHS = 30 # maximum training epochs
LEARNING_RATE = 0.001 # Adam optimizer learning rate

# — Load data —————
print('Loading data...')
df = pd.read_csv(DATA_PATH)
df['seance'] = pd.to_datetime(df['seance'])
df['direction_7d'] = df['direction_7d'].astype(str) # TFT needs string for classification

print(f'Loaded: {len(df):,} rows, {df["ticker_id"].nunique()} stocks')

# — Train/validation split —————
# Use 2016-2023 for training, 2024 for validation, 2025 for test
```



```

# This is walk-forward validation – no future data leaks into training
cutoff_train = df['seance'] < '2024-01-01'
cutoff_val = (df['seance'] >= '2024-01-01') & (df['seance'] < '2025-01-01')

max_train_idx = df[cutoff_train]['time_idx'].max()

print(f'Training rows : {cutoff_train.sum():,} (2016-2023)')
print(f'Validation rows: {cutoff_val.sum():,} (2024)')

# — Build TimeSeriesDataSet —————
# TimeSeriesDataSet is pytorch-forecasting's data format for TFT
# It handles the sliding window, batching, and normalization automatically
training = TimeSeriesDataSet(
    df[cutoff_train],
    time_idx = 'time_idx',
    target = 'direction_7d',
    group_ids = ['ticker_id'],
    min_encoder_length = MAX_ENCODER_LENGTH // 2,
    max_encoder_length = MAX_ENCODER_LENGTH,
    min_prediction_length = 1,
    max_prediction_length = MAX_PREDICTION_LENGTH,
    static_categoricals = ['ticker_id', 'company_type_id'],
    time_varying_known_reals = ['time_idx'],
    time_varying_unknown_reals = [
        'close_price', 'volume', 'daily_return',
        'ma_5', 'ma_20', 'ma_50', 'rsi_14',
        'volatility_20', 'volume_ma_20',
    ],
    target_normalizer = None, # classification target – no normalization
    add_relative_time_idx = True,
    add_target_scales = False,
    add_encoder_length = True,
)

validation = TimeSeriesDataSet.from_dataset(
    training, df[cutoff_val | cutoff_train],
    predict=True, stop_randomization=True
)

train_dl = training.to_dataloader(train=True, batch_size=BATCH_SIZE, num_workers=0)
val_dl = validation.to_dataloader(train=False, batch_size=BATCH_SIZE, num_workers=0)

# — Build TFT model —————
tft = TemporalFusionTransformer.from_dataset(
    training,
    learning_rate = LEARNING_RATE,
    hidden_size = 64, # model width – increase to 128 if GPU allows
    attention_head_size = 4, # multi-head attention
    dropout = 0.1, # regularization
    hidden_continuous_size = 32,
    loss = CrossEntropy(), # binary classification loss
    log_interval = 10,
    reduce_on_plateau_patience = 3,
)

print(f'TFT parameters: {tft.size()/1e3:.1f}K')

# — Callbacks —————
early_stop = EarlyStopping(

```

```

    monitor = 'val_loss',
    patience = 5,          # stop if no improvement for 5 epochs
    mode     = 'min'
)
checkpoint = ModelCheckpoint(
    dirpath   = str(MODEL_DIR),
    filename  = 'tft_bvmt_{epoch:02d}_{val_loss:.4f}',
    monitor   = 'val_loss',
    mode      = 'min',
    save_top_k = 1          # keep only the best model
)

# — Train —————
trainer = Trainer(
    max_epochs      = MAX_EPOCHS,
    accelerator     = 'gpu' if torch.cuda.is_available() else 'cpu',
    gradient_clip_val = 0.1,
    callbacks       = [early_stop, checkpoint],
    enable_progress_bar = True,
)

print('Starting training...')
trainer.fit(tft, train_dl, val_dl)

print(f'Best model saved to: {checkpoint.best_model_path}')
print('Training complete!')

```

Part B — Set up Lightning AI Studio (30 minutes)

Lightning AI gives you a free T4 GPU (16GB VRAM) for model training. Your laptop never needs a GPU.

#	Action	What to do	Expected result
1	Create account	Go to lightning.ai → sign up with student email	Free account created
2	New Studio	Click New Studio → choose Python environment → click Create	Studio starts in ~60 seconds
3	Open Terminal	Inside the studio, click the Terminal icon at the bottom	Bash terminal opens
4	Install packages	Run: <code>pip install pytorch-forecasting pytorch-lightning</code>	Takes ~3 minutes
5	Create folders	Run: <code>mkdir -p data/features models training</code>	Folder structure ready
6	Upload CSV	Drag <code>tft_features.csv</code> into the <code>data/features/</code> folder in the file browser	~30-60 seconds upload
7	Upload script	Drag <code>train_tft.py</code> into the <code>training/</code> folder	File appears in browser
8	Verify GPU	In terminal: <code>python -c "import torch; print(torch.cuda.is_available())"</code>	Should print True

If `torch.cuda.is_available()` prints False

The studio may have started in CPU mode. Click the GPU button in the top bar.
Select T4 GPU from the dropdown and click Apply.
Wait 30 seconds for it to restart, then check again.

DAY 13 — Train the TFT Model on Lightning AI (3-4 hours)

Day 13 is the exciting day — you run the actual training. The T4 GPU will process your 120,000+ rows and learn to predict stock direction. You do not need to watch it — start the training, do something else for 1-2 hours, then come back to check results.

Part A — Start training (10 minutes of work, 1-2 hours waiting)

Step 1 — Open your Lightning AI studio

Go to lightning.ai, open the studio you created on Day 12. Confirm the GPU is active (green GPU icon in top bar).

Step 2 — Run the training script

In the terminal inside Lightning AI, run:

```
cd /path/to/your/studio # whatever your working directory is
python training/train_tft.py
```

You will see output like this — this means it is working correctly:

```
Loading data...
Loaded: 115,432 rows, 65 stocks
Training rows : 87,210 (2016-2023)
Validation rows: 14,203 (2024)
TFT parameters: 847.3K
Starting training...
Epoch 1/30: 100%|██████| loss=0.693 val_loss=0.681
Epoch 2/30: 100%|██████| loss=0.672 val_loss=0.665
...
```

Step 3 — What good training looks like

Watch the `val_loss` for the first 5 epochs. Here is how to interpret it:

val_loss pattern	What it means	What to do
0.693 → 0.680 → 0.670	Learning normally ✓	Let it run — this is perfect
0.693 → 0.693 → 0.693	Not learning at all	Stop. Learning rate too low. Change to 0.003
0.693 → 0.750 → 0.800	Getting worse (diverging)	Stop. Learning rate too high. Change to 0.0003
Stops at epoch 8	Early stopping triggered ✓	Normal — model converged, best weights saved

val_loss starts at ~0.693 because that is the random guess baseline for binary classification ($\log(2) \approx 0.693$). Any value below 0.680 after 5 epochs means the model is learning something real.

Part B — Download the model file (after training finishes)

When training finishes you will see:

```
Best model saved to: models/tft_bvmt_epoch=14_val_loss=0.6421.ckpt
Training complete!
```

Download the .ckpt file from Lightning AI to your local machine:

- In the Lightning AI file browser, right-click the .ckpt file
- Click Download
- Save it to your local project at:

```
C:\Users\Negza\Desktop\projects\pfe\bvmt_project\models\tft_bvmt_best.ckpt
```

Part C — Evaluate accuracy (30 minutes)

After downloading the model, create a new notebook locally: notebooks/evaluate_tft.ipynb

```
import pandas as pd
import torch
from pytorch_forecasting import TemporalFusionTransformer, TimeSeriesDataSet
from pathlib import Path

# Load the saved model
model_path = Path('models/tft_bvmt_best.ckpt')
tft = TemporalFusionTransformer.load_from_checkpoint(str(model_path))
tft.eval()

# Load 2025 test data (out-of-sample - model never saw this)
df = pd.read_csv('data/features/tft_features.csv')
df['seance'] = pd.to_datetime(df['seance'])
df['direction_7d'] = df['direction_7d'].astype(str)

# Use 2025 data for final test
df_test = df[df['seance'] >= '2025-01-01']
print(f'Test rows (2025): {len(df_test):,}')

# Quick accuracy check using predictions
# (Full evaluation with proper sliding windows comes in Sprint 6)
print('Model loaded successfully - ready for TechnicalAgent integration')
```

Part D — What target accuracy means for your thesis

Accuracy	What it means	Thesis interpretation
----------	---------------	-----------------------

< 50%	Worse than random	Model failed — check data leakage or use more features
50–55%	Barely better than random	Acceptable for thesis — markets are hard to predict
55–65%	Meaningful signal ✓	Good result — cite as evidence of TFT effectiveness on BVMT
> 65%	Strong result ✓	Excellent — likely means BVMT has exploitable momentum patterns

Important note for your thesis

Do NOT aim for very high accuracy (>75%). That usually means data leakage.
 Data leakage = the model accidentally sees future information during training.
 55-65% accuracy with a clean walk-forward validation is academically strong.
 The comparison TFT vs CNN-LSTM matters more than the absolute number.

Before You Start Day 11 — One Remaining Task

Tomorrow when you open your project, finish the ONE TECH HOLDING repair before starting Day 11. It uses only 5 API requests and takes 5 minutes:

```
# In your repair notebook — add a new cell and run it
for fname, period, ped in [
    ('oth_efd311218.pdf', 'FY 2018', '2018-12-31'),
    ('oth_efd311220.pdf', 'FY 2020', '2020-12-31'),
    ('oth_efd311221.pdf', 'FY 2021', '2021-12-31'),
    ('oth_efd311222.pdf', 'FY 2022', '2022-12-31'),
    ('oth_efd311224.pdf', 'FY 2024', '2024-12-31'),
]:
    repair_pdf('ONE TECH HOLDING', fname, period, ped,
              unit='DT', company_type='non_bank',
              page_indices=[6, 7, 8, 9, 10],
              isin_map=isin_map)
```

After that runs, proceed to `feature_engineering.ipynb` for Day 11.